

Universidad Galileo	Guatemala 05 de Octubre del 2026
Facultad: FISICC	Alumno: Roberto Molina
Curso: Microprocesadores	Carnet: 24000849
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Mendoza	Día de Laboratorio: Lunes

Laboratorio #7 Puertos Paralelos en Lenguaje Ensamblador

Resumen:

En la práctica de laboratorio se trabajó nuevamente con los puertos paralelos GPIO del microcontrolador STM32G431, pero esta vez programando la configuración y el control del LED en lenguaje ensamblador Thumb-2, utilizando Visual Studio Code. El objetivo principal fue aprender a desarrollar subrutinas en ensamblador e integrarlas con un programa escrito en lenguaje C.

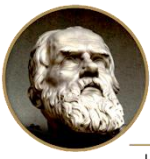
El proyecto base incluía dos archivos: main.c y GPIO.s. En main.c se declaran con el modificador extern las funciones LEDs_Init y LED_Toggle, que se implementan en GPIO.s y se hacen visibles para el resto del proyecto con la directiva EXPORT. La función main configura el temporizador SysTick, llama a LEDs_Init y luego se queda en un ciclo infinito. En cada interrupción del SysTick se llama a LED_Toggle, replicando el funcionamiento de la práctica de laboratorio #4. En main.c solo se cambió el divisor del SysTick de 50 a 10.

Para completar GPIO.s fue necesario consultar el manual de referencia del microcontrolador y definir correctamente los símbolos con las direcciones base del RCC y de los puertos GPIO, así como los offsets de los registros de configuración, ya que el código proporcionado traía valores de ejemplo de otras tarjetas.

Finalmente, se implementaron las subrutinas LEDs_Init, que habilita el reloj del puerto y configura el pin del LED como salida, y LED_Toggle, que invierte el estado del LED. Para esto se aplicó la secuencia de lectura, modificación y escritura con instrucciones como LDR, STR, ORR, BIC y EOR.

Objetivos:

- Desarrollar subrutinas en lenguaje ensamblador Thumb-2 para el microcontrolador STM32G431 que configuren un pin GPIO como salida y controlen su estado.



- Integrar código escrito en C con código escrito en ensamblador, llamando desde main.c a subrutinas definidas en GPIO.s mediante el modificador extern y la directiva EXPORT.
- Consultar el manual de referencia para definir las direcciones base y los offsets de los registros del RCC y del GPIO, y manipularlos con instrucciones de carga, almacenamiento y operaciones lógicas a nivel de bits.

Teoría:

Lenguaje ensamblador y set de instrucciones Thumb-2

El lenguaje ensamblador es una representación legible del código de máquina: cada instrucción corresponde, en general, a una instrucción que ejecuta directamente el CPU. A diferencia de C, el programador controla directamente los registros del procesador y los accesos a memoria. El núcleo Cortex-M4 del STM32G431 implementa la arquitectura ARMv7-M y solo ejecuta instrucciones del set Thumb, en su versión Thumb-2, que combina instrucciones de 16 y de 32 bits para lograr un código compacto sin perder rendimiento [1].

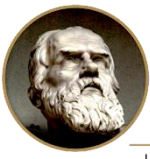
El Cortex-M4 tiene una arquitectura de tipo load/store: las operaciones aritméticas y lógicas solo trabajan sobre registros del procesador, por lo que para modificar un registro de un periférico primero hay que cargar su valor a un registro, operar sobre él y luego volver a guardarlo en memoria [1].

Registros del procesador

El procesador cuenta con 13 registros de propósito general de 32 bits (R0 a R12) y tres registros especiales: R13 o SP (Stack Pointer), que apunta al tope de la pila, R14 o LR (Link Register), que guarda la dirección de retorno cuando se llama a una subrutina con la instrucción BL y R15 o PC (Program Counter), que contiene la dirección de la siguiente instrucción a ejecutar [1].

Integración de C con ensamblador

El modificador extern indica que una declaración hace referencia a un símbolo definido en otro archivo fuente. No crea la definición, solo le avisa al compilador que existe [2]. En el lado del ensamblador, la directiva EXPORT declara un símbolo que puede ser usado por otros archivos [3]. Al momento de enlazar, el linker une las llamadas hechas desde C con las etiquetas exportadas en el archivo .s, por lo que ambos lenguajes pueden convivir en el mismo proyecto.



Para que esto funcione, el código en ensamblador debe respetar la convención de llamadas de Arm (AAPCS). Según esta convención, los registros R0 a R3 se usan para pasar argumentos y R0 para el valor de retorno, estos registros, junto con R12, pueden ser modificados libremente por la subrutina. En cambio, los registros R4 a R11 deben conservar su valor al regresar, y la pila debe mantenerse alineada a 8 bytes en las interfaces públicas [4]. Por eso las subrutinas del laboratorio inician con `push {lr}`, que guarda la dirección de retorno en la pila, y terminan con `pop {pc}`, que la carga directamente en el contador de programa para regresar a la función que hizo la llamada.

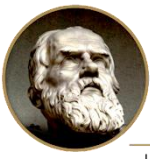
Directivas del ensamblador

Las directivas no generan instrucciones de máquina, sino que le dan indicaciones al ensamblador. Las utilizadas en GPIO.s son [3]:

- **AREA:** define una sección del programa, en este caso de código (CODE) de solo lectura (READONLY) y alineada a 4 bytes (ALIGN=2).
- **THUMB:** indica que las instrucciones siguientes se ensamblan con el set Thumb.
- **EXPORT:** hace visible un símbolo para los demás archivos del proyecto, como LEDs_Init y LED_Toggle.
- **EQU:** le asigna un nombre a una constante numérica, como una dirección base o un offset. No ocupa memoria, el ensamblador sustituye el nombre por su valor.
- **PRESERVE8 / REQUIRE8:** indican que el código mantiene y requiere que la pila esté alineada a 8 bytes, como lo pide la AAPCS.
- **END:** marca el final del archivo fuente.

Instrucciones utilizadas

- **LDR Rt, [Rn, #offset]:** carga en Rt la palabra de 32 bits que se encuentra en la dirección Rn + offset [1]. La pseudoinstrucción `LDR Rd, =constante` permite cargar una constante de 32 bits, como la dirección base de un periférico, que el ensamblador coloca en un literal pool [3].
- **STR Rt, [Rn, #offset]:** guarda el contenido de Rt en la dirección Rn + offset [1].
- **MOVS Rd, #inm:** carga un valor inmediato en Rd. El sufijo S indica que la instrucción actualiza las banderas de estado [1].



- **ORR, BIC y EOR:** realizan operaciones OR, AND con el complemento (bit clear) y XOR bit a bit entre registros [1]. Con una máscara adecuada sirven para poner bits en 1, ponerlos en 0 o invertirlos, respectivamente.
- **PUSH y POP:** guardan y recuperan registros de la pila [1].

Mapa de memoria y direcciones de los periféricos

En el STM32G4 los periféricos están mapeados en memoria: cada módulo tiene una dirección base y cada uno de sus registros se encuentra a un desplazamiento (offset) fijo de esa base, de modo que la dirección de un registro es la suma de ambos valores [5]. Las direcciones relevantes para esta práctica son [5]:

- **RCC:** dirección base 0x4002 1000. El registro RCC_AHB2ENR, que habilita el reloj de los puertos GPIO, está en el offset 0x4C. El bit 0 corresponde a GPIOAEN y el bit 1 a GPIOBEN.
- **GPIOA:** dirección base 0x4800 0000.
- **GPIOB:** dirección base 0x4800 0400.
- **GPIOx_MODER:** offset 0x00. **GPIOx_ODR:** offset 0x14.

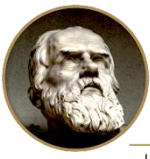
Los valores que aparecen como ejemplo en el código base (AHBENR en el offset 0x14 y el bit 18) corresponden a otra familia de microcontroladores, en la que los puertos GPIO están en el bus AHB. En el STM32G4 los puertos están conectados al bus AHB2, por lo que su reloj se habilita en RCC_AHB2ENR [5]. En la tarjeta NUCLEO-G431KB, el LED de usuario LD2 está conectado al pin PB8 [6].

Secuencia de lectura, modificación y escritura

Como el procesador no puede operar directamente sobre la memoria, cambiar algunos bits de un registro de periférico requiere tres pasos: leer el registro con LDR, modificar el valor con una operación lógica y una máscara (ORR para encender bits, BIC para apagarlos o EOR para invertirlos) y escribir el resultado con STR [1]. Esta secuencia es la que el compilador genera para expresiones en C como `GPIOx->MODER |= máscara`, y permite modificar solo los bits necesarios sin alterar la configuración de los demás pines.

Temporizador SysTick

El SysTick es un temporizador de 24 bits incluido en el núcleo Cortex-M4. Cuenta de forma descendente desde un valor de recarga y, al llegar a cero, se recarga automáticamente y



puede generar una excepción [1]. La función de CMSIS SysTick_Config(ticks) configura el valor de recarga, habilita el contador y su interrupción, y devuelve 0 si la configuración fue exitosa [7]. Cada vez que ocurre la interrupción se ejecuta la rutina SysTick_Handler.

En el proyecto, SystemCoreClock vale 16 MHz, que es la frecuencia del oscilador interno HSI con el que arranca el microcontrolador. Por lo tanto, SysTick_Config(SystemCoreClock/10) genera una interrupción cada:

$$T = \frac{ticks}{f_{CLK}} = \frac{1\,600\,000}{16\,000\,000\,Hz} = 100\,ms$$

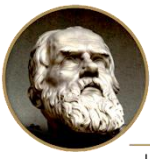
Es decir, 10 interrupciones por segundo. Como el LED cambia de estado en cada interrupción, un ciclo completo de encendido y apagado dura 200 ms.

Bit banding

El Cortex-M4 ofrece una técnica llamada bit banding, que asigna a cada bit de ciertas regiones de memoria una palabra completa en otra región llamada alias. Escribir un 1 o un 0 en esa palabra del alias pone en 1 o en 0 el bit correspondiente en una sola operación, sin necesidad de hacer la secuencia de lectura, modificación y escritura [1]. Para los periféricos, la región de bit banding va de 0x4000 0000 a 0x400F FFFF y su alias empieza en 0x4200 0000 [1]. La dirección del alias se calcula como:

$$\text{Dirección alias} = 0x42000000 + (\text{offset} \times 32) + (n \times 4)$$

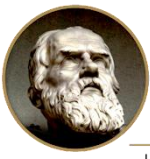
donde offset es la distancia en bytes entre el registro y 0x4000 0000, y n es el número de bit [1]. El RCC (0x4002 1000) se encuentra dentro de esta región, por lo que sus registros se pueden modificar bit a bit de esta forma. En cambio, los puertos GPIO del STM32G4 están en 0x4800 0000, fuera de la región de bit banding [5].



Resultados prácticos:

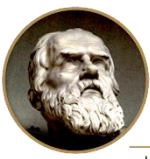
```
G4_Test1 > [w] GPIO.s
1      AREA |.text|, CODE, READONLY, ALIGN=2
2      THUMB
3      REQUIRES
4      PRESERVE8
5
6
7      EXPORT  LEDs_Init
8      EXPORT  LED_Toggle
9
10     RCC      EQU      0x40021000
11
12     ; Deben buscar el valor en el Reference Manual
13     GPIOE     EQU      0x48001000 ; EvalBoard
14     ;GPIOB     EQU      0x48000400 ; K8
15     GPIOB     EQU      0x48000400 ; G4
16     GPIOA     EQU      0x48000000 ; RE
17
18     AHB2ENR    EQU      0x4C
19
20     ; Deben buscar el valor en el Reference Manual
21     MODER      EQU      0x00
22     ODR         EQU      0x14
23
24     ; Bit Banding puntos extras
25     ; alias = BitBanding_alias + (offset del byte dentro de la region * 32) + (bit * 4)
26     BitBand     EQU      0x40000000 ; inicio de la region bit-band de perifericos
27     BitBand_ALIAS EQU      0x42000000 ; inicio de la region alias de perifericos
28     Bit_N       EQU      18         ; modificar el bit n
29
30     AHB2ENR_BitBand EQU      BitBand_ALIAS + ((RCC + AHB2ENR - BitBand) * 32) + (Bit_N * 4)
31
```

Fig. 1. Declaración de constantes



```
33 LEDs_Init
34     push {lr}
35
36     ;RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; (bit 1)
37     ;**** Encender el puerto correspondiente de tu tarjeta ****
38
39     ldr r0, =RCC           ;read
40     ldr r1, [r0, #AHB2ENR]
41     movs r2, #0x2          ;modify: bit 1 = GPIOBEN
42     orrs r1, r1, r2
43     str r1, [r0, #AHB2ENR] ;write
44
45     ;EvalBoard / Discovery
46     ; GPIOE->MODER |= (1<<26);
47     ; GPIOE->ODR &= ~(1<<13);
48     ;F303K8
49     ; GPIOB->MODER &= ~(1<<7);
50     ; GPIOB->MODER |= (1<<6);
51     ;F303RE
52     ; GPIOA->MODER |= (1<<10);
53     ; GPIOA->ODR &= ~(1<<5);
54     ;G4 (NUCLEO-G431KB) - LED en PB8
55     ; GPIOB->MODER &= ~(1<<17);
56     ; GPIOB->MODER |= (1<<16);
57     ; GPIOB->ODR &= ~(1<<8);
58
59     ldr r0, =GPIOB         ;read
60     ldr r1, [r0, #MODER]
61     movs r2, #0x20000      ;modify clear bit 17
62     bics r1, r1, r2
63     movs r2, #0x10000      ;modify set bit 16 -> MODER[17:16] = 01
64     orrs r1, r1, r2
65     str r1, [r0, #MODER]   ;write
66
67     ldr r1, [r0, #ODR]     ;read
68     movs r2, #0x100        ;modify: clear bit 8 LED apagado al inicio
69     bics r1, r1, r2
70     str r1, [r0, #ODR]     ;write
71
72     ;Bit Banding: RCC->AHB2ENR |= (1<<Bit_N);
73     ;Cada bit del registro tiene su propia palabra de 32 bits en la region alias,
74     ;escribir 1 en esa palabra pone en 1 solo ese bit sin read-modify-write
75
76     ldr r0, =AHB2ENR_BitBand ;direccion alias del bit n
77     movs r1, #1
78     str r1, [r0]           ;write bit n = 1
79     pop {pc}
```

Fig. 2. Subrutina LEDs_Init



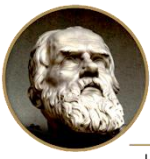
```
81  ▾ LED_Toggle
82      push {lr}
83
84      ;GPIOE->ODR ^= 1<<13 EvalBoard / Discovery
85      ;GPIOB->ODR ^= 1<<3 K8
86      ;GPIOA->ODR ^= 1<<5 RE
87      ;GPIOB->ODR ^= 1<<8 G4
88
89      ldr  r0, =GPIOB      ;read
90      ldr  r1, [r0,#ODR]
91      movs r2, #0x100      ;modify
92      eors r1, r1, r2
93      str  r1, [r0,#ODR] ;write
94
95      pop {pc}
96
97      end
98
```

Fig. 3. Subrutina LED_Toggle



Fig. 4. Tarjeta funcionando

Video demostrativo: https://www.youtube.com/shorts/Rt_Qljar_u8



Procedimientos teóricos:

Preguntas de la guía

¿Por qué las funciones declaradas en main.c utilizan el modificador extern?

Porque LEDs_Init y LED_Toggle no están escritas en main.c, solo se declaran ahí. El modificador extern le indica al compilador que la definición de esas funciones se encuentra en otro archivo fuente. Así, el compilador puede generar las llamadas en main() y en SysTick_Handler() sin conocer todavía su código, y es el enlazador el que, al final, une cada llamada con la dirección real de la subrutina. En C las funciones ya son externas por defecto, pero escribir extern deja claro que el código está en otro archivo.

¿Dónde están definidas esas subrutinas?

En el archivo GPIO.s. Cada subrutina es una etiqueta (LEDs_Init y LED_Toggle) seguida de sus instrucciones en Thumb-2. Al inicio del archivo se usa la directiva EXPORT con ambos nombres, para que los símbolos sean visibles fuera de GPIO.s y el enlazador los pueda conectar con las declaraciones extern de main.c.

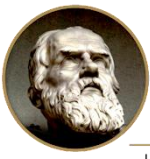
¿Ya se dio cuenta de que estamos replicando el Laboratorio #4?

Sí. La aplicación es la misma: se habilita el reloj del puerto, se configura el pin del LED como salida y luego el LED cambia de estado en cada interrupción del SysTick. La diferencia es que las líneas que antes se escribían en C, como `GPIOB->MODER |= (1<<16)` o `GPIOB->ODR ^= (1<<8)`, ahora se escriben como una secuencia de instrucciones en ensamblador. Por eso en GPIO.s se dejaron como comentario las líneas equivalentes en C.

Definición de símbolos (Fig. 1)

Los símbolos del archivo GPIO.s se definieron con la directiva EQU, usando los valores del manual de referencia:

- **RCC = 0x40021000:** dirección base del módulo de reloj.
- **GPIOA = 0x48000000, GPIOB = 0x48000400 y GPIOE = 0x48001000:** direcciones base de los puertos. Se usa GPIOB porque el LED de usuario LD2 de la NUCLEO-G431KB está conectado al pin PB8.
- **AHB2ENR = 0x4C:** offset del registro RCC_AHB2ENR. Se cambió el nombre del símbolo original (AHBENR = 0x14), porque ese valor corresponde al registro AHBENR de la familia STM32F3 y no existe en el STM32G4.
- **MODER = 0x00 y ODR = 0x14:** offsets de los registros de modo y de datos de salida, iguales en todos los puertos.



El código base definía dos veces el símbolo GPIOB (una para la F303K8 y otra para la G4). Como un símbolo definido con EQU no puede tener dos valores, la línea de la K8 se dejó como comentario para evitar un error al ensamblar.

Subrutina LEDs_Init (Fig. 2)

La subrutina inicia con push {lr} para guardar la dirección de retorno y termina con pop {pc} para regresar a main(), como lo pide la convención de llamadas. Todo el trabajo se hace con los registros R0, R1 y R2, que la subrutina puede modificar libremente.

Habilitación del reloj. Primero se carga en R0 la dirección base del RCC con ldr r0, =RCC, y luego se lee el registro con ldr r1, [r0, #AHB2ENR]. En R2 se carga la máscara 0x2, que corresponde al bit 1 (GPIOBEN), se aplica un OR con orrs r1, r1, r2 y el resultado se guarda con str r1, [r0, #AHB2ENR]. Es el equivalente en ensamblador de $RCC \rightarrow AHB2ENR |= RCC_AHB2ENR_GPIOBEN$.

Configuración de PB8 como salida. El pin 8 usa los bits 17 y 16 del registro MODER, y para que sea salida deben quedar en 01. Se carga la dirección de GPIOB en R0 y se lee MODER en R1. Luego se limpia el bit 17 con la máscara 0x20000 y la instrucción bics, y se pone en 1 el bit 16 con la máscara 0x10000 y la instrucción orrs. Por último, se escribe el resultado con str. Se limpia antes de escribir para asegurar el valor 01 sin importar cómo estaba el registro.

Estado inicial del LED. Se lee el registro ODR, se limpia el bit 8 con la máscara 0x100 y bics, y se vuelve a escribir. De esta forma el LED empieza apagado.

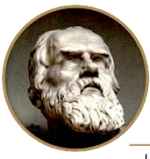
Bit banding (Fig. 1 y Fig. 2)

Para los puntos extra se modificó el bit $n = 18$ del registro $RCC_AHB2ENR$ usando bit banding. En GPIO.s se definieron los símbolos BitBand = 0x40000000 (inicio de la región de periféricos) y BitBand_ALIAS = 0x42000000 (inicio de su región alias), y se calculó la dirección alias con la fórmula vista en la teoría:

$$Alias = 0x42000000 + (0x4002104C - 0x40000000) \times 32 + 18 \times 4$$

$$Alias = 0x42000000 + 0x420980 + 0x48 = 0x424209C8$$

Como este cálculo se hace con símbolos EQU, el ensamblador obtiene el valor al momento de ensamblar y no se gastan instrucciones en ejecución. En LEDs_Init se carga esa dirección con ldr r0, =AHB2ENR_BitBand, se pone R1 en 1 con movs r1, #1 y se escribe con str r1, [r0]. Esa única escritura pone en 1 solo el bit 18 de $RCC_AHB2ENR$, sin tener que

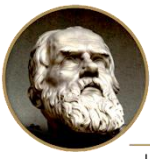


leer el registro, aplicar la máscara y volver a escribir. El bit 18 corresponde al reloj del DAC3, por lo que encenderlo no afecta el funcionamiento del LED.

Subrutina LED_Toggle (Fig. 3)

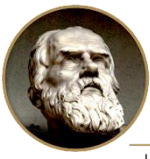
Para cambiar el estado del LED se usa una operación XOR, ya que al hacer XOR de un bit con 1 su valor se invierte, y al hacerlo con 0 se mantiene. Se carga la dirección de GPIOB en R0, se lee ODR en R1, se carga la máscara 0x100 (bit 8) en R2 y se aplica `eors r1, r1, r2`. Al escribir el resultado con `str`, solo cambia el pin PB8 y el resto del puerto queda igual. Es el equivalente de `GPIOB->ODR ^= (1<<8)`.

Esta subrutina se llama desde SysTick_Handler cada 100 ms, ya que en main.c el SysTick se configuró con `SystemCoreClock/10`. Por lo tanto, el LED cambia de estado 10 veces por segundo, es decir, parpadea a 5 Hz. En la Fig. 4 se observa el LED LD2 de la tarjeta funcionando con el programa.



Conclusiones:

- Se pudo observar que programar en ensamblador la misma aplicación del Laboratorio #4 requiere más líneas de código y más atención a los detalles, ya que una sola instrucción en C, como `GPIOx->ODR ^= máscara`, se convierte en una secuencia de carga, operación lógica y almacenamiento. Esto ayudó a entender qué es lo que realmente hace el compilador con el código en C.
- Es muy importante consultar el manual de referencia, porque las direcciones y los bits del código base correspondían a otro microcontrolador. Un valor incorrecto en un símbolo EQU no genera ningún error al compilar, pero hace que el programa escriba en un registro equivocado y el LED simplemente no funcione.
- Cuando el programa ya funcionaba con la configuración original del SysTick, el LED parpadeaba tan rápido que casi parecía estar encendido todo el tiempo. Al cambiar el divisor a `SystemCoreClock/10`, el parpadeo se volvió claramente visible. Esto ayudó a entender en la práctica la relación entre el valor de recarga del SysTick y la frecuencia con la que se ejecuta la interrupción.



Bibliografía:

- [1] STMicroelectronics, *STM32 Cortex-M4 MCUs and MPUs Programming Manual*, PM0214. Accedido el 7 de octubre de 2026. [En línea]. Disponible: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf
- [2] Microsoft, "extern (Especificador de clase de almacenamiento)," Microsoft Learn. Accedido el 7 de octubre de 2026. [En línea]. Disponible: <https://learn.microsoft.com/es-es/cpp/c-language/extern-storage-class-specifier?view=msvc-170>
- [3] Arm Ltd., *Arm Compiler armasm User Guide*, DUI0801. Accedido el 7 de octubre de 2026. [En línea]. Disponible: <https://developer.arm.com/documentation/dui0801/latest/>
- [4] Arm Ltd., *Procedure Call Standard for the Arm Architecture (AAPCS32)*. Accedido el 7 de octubre de 2026. [En línea]. Disponible: <https://github.com/ARM-software/abi-aa/blob/main/aapcs32/aapcs32.rst>
- [5] STMicroelectronics, *STM32G4 Series Advanced Arm-Based 32-bit MCUs*, RM0440. Accedido el 7 de octubre de 2026. [En línea]. Disponible: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [6] STMicroelectronics, *STM32G4 Nucleo-32 Board (MB1430)*, UM2397, Rev. 2, sep. 2019. Accedido el 7 de octubre de 2026. [En línea]. Disponible: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf
- [7] Arm Ltd., "CMSIS-Core (Cortex-M): SysTick Timer (SYSTICK)," Arm Software. Accedido el 7 de octubre de 2026. [En línea]. Disponible: https://arm-software.github.io/CMSIS_6/latest/Core/group_SysTick_gr.html